《大模型与智能体技术》实验报告单

姓名：郑茂繁　学号：26053040　日期：2026 年 11 月 3 日

一、实验环境记录

硬件环境：搭载 AMD Ryzen7 7800X3D 八核十六线程处理器（4.2GHz，96MBL3 缓存，AM5 平台）、24GB DDR5 内存（单条 24GB，EXPO 已开启、实际运行频率 6000MT/s）与 NVIDIA GeForce RTX5060Ti16GB 显卡（驱动 621.01，CUDA13.5，TDP180W，另含 AMDRadeon 核显），主板为 ASUS B650M-AYW WIFI（BIOS3602）；存储方面 C 盘 96.7GB（剩余 9.6GB）、D 盘 200GB（剩余 167.6GB）、E 盘 200GB（剩余 161.2GB）、F 盘 433.6GB（剩余 188.9GB），显示分辨率 1920×1080；该配置的 16GB 显存可完整容纳 Qwen3-8BQ4_K_M 量化模型（约 4.7GB）及 32K 上下文缓存，实测推理速度约 76tok/s、首字延迟约 150ms，满足本实验中本地大模型部署、对话调用与流式输出的需求，同时在学习的过程中使用 deepseek-flash 进行辅助开发。

软件环境：Windows11 专业版+Python3.12（项目内.venv 虚拟环境）+llama.cppCUDA 预编译推理引擎（llama-server，build11946，CUDA13.4 版）+GGUF 格式的 Qwen3-8B-Q4_K_M 量化模型（约 4.7GB），由 llama-server 对外提供 OpenAI 兼容 HTTP 接口（127.0.0.1:8010），Python 侧仅依赖 openai/requests/numpy 等轻量包负责启动、调用与实验脚本，不依赖 PyTorch、Transformers 等深度学习框架。

开发与依赖：Python3.12（项目内.venv）；调用用 openai、requests、numpy、Pillow、matplotlib、scipy；智能体工具 DeepSeek Harness；LoRA 微调用 torch、transformers、peft、trl（CPU 版）；Node.jsLTS 用于工具链实验。

二、大模型使用及调用的实践

【实验要求（实践一：大模型的使用及调用）】

1. 完成 Ollama 安装，成功拉取并运行 qwen3.5:0.8b 模型；

2. 通过浏览器或 curl 验证本地服务可用；

3. 安装并配置 Chatbox，完成与本地模型的图形化对话；

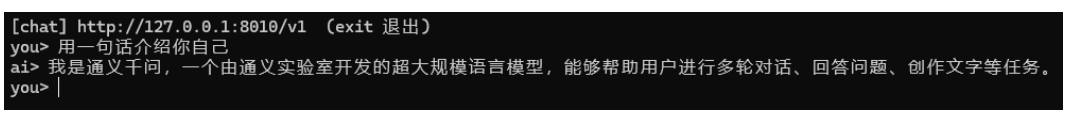
4. 使用 Python SDK 完成同步与流式两种调用；

5. 运行 chat_loop.py 实现多轮对话，截图记录运行结果。

说明：这里使用的是 llama 而不是 ollama 部署本地模型，所以部分实验步骤不能提供对应证明，所以第 1 条用 llama.cpp 运行 Qwen3-8B 的 Q4_K_M 量化版本代替。

【实验要求完成情况】

1. 完成 Ollama 安装，成功拉取并运行 qwen3.5:0.8b 模型：



```
============================================================
  Local model only (llama.cpp, no proxies)
  Endpoint: http://127.0.0.1:8010/v1
  Stop    : Ctrl+C
============================================================

[cmd] E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\Qwen3-7B\runtime\llama\llama-server.exe -m E:\Code\
Graduate-course-code\Large-Models-and-Agent-Technology\Qwen3-7B\models\Qwen3-8B-Q4_K_M.gguf --host 127.0.0.1 --port 8010
 -c 32768 -ngl 99 -t 8 -b 1024 -ub 256 --metrics --jinja --reasoning off
[serve] OpenAI base_url = http://127.0.0.1:8010/v1
[serve] 模型 = Qwen3-8B-Q4_K_M.gguf  ctx = 32768  ngl = 99
0.00.084.740 I srv  llama_server: initializing ...
0.00.084.747 I cmn  common_param: common_params_print_info: verbosity = 3 (adjust with the `-lv N` CLI arg)
0.00.085.012 W srv  server_setup: security: no API key is set and CORS allows all origins (see https://github.com/ggml-o
rg/llama.cpp/pull/25655)
0.00.096.146 I srv    load_model: loading model 'E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\Qwen3-7B
\models\Qwen3-8B-Q4_K_M.gguf'
0.00.327.592 W load: control-looking token: 128247 '</s>' was not control-type; this is probably a bug in the model. its
 type will be overridden
0.04.805.402 I cmn          init: llama threadpool init, n_threads = 8
0.04.949.531 I srv    load_model: initializing, n_slots = 4, n_ctx_slot = 32768, kv_unified = 'true'
0.04.957.196 I srv  server_run_i: model loaded
0.04.957.203 I srv  server_on_li: listening on http://127.0.0.1:8010
```

2. 通过浏览器或 curl 验证本地服务可用；

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

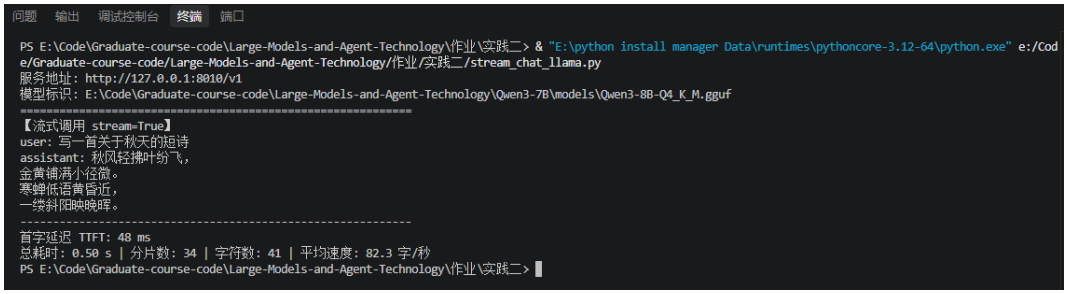
PS E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\Qwen3-7B\scripts> curl.exe -s http://127.0.0.1:8010/health
{"status":"ok"}
PS E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\Qwen3-7B\scripts> curl.exe -s http://127.0.0.1:8010/v1/models
{"models":[{"name":"E:\\Code\\Graduate-course-code\\Large-Models-and-Agent-Technology\\Qwen3-7B\\models\\Qwen3-8B-Q4_K_M.gguf","model":"E:\\Code\\Graduate-course-code\\Large-Models-and-Agent-Technology\\Qwen3-
7B\\models\\Qwen3-8B-Q4_K_M.gguf","modified_at":"","size":"","digest":"","type":"model","description":"","tags":[""],"capabilities":["completion"],"parameters":"","details":{"parent_model":"","format":"gguf","
family":"","families":[""],"parameter_size":"","quantization_level":""}}],"object":"list","data":[{"id":"E:\\Code\\Graduate-course-code\\Large-Models-and-Agent-Technology\\Qwen3-7B\\models\\Qwen3-8B-Q4_K_M.gguf","aliases":["E:\\Code\\Graduate-course-code\\Large-Models-and-Agent-Technology\\Qwen3-7B\\models\\Qwen3-8B-Q4_K_M.gguf"],"tags":[],"object":"model","architecture":{"input_modalities":["text"],"output_modalities":["text"]},"created":1793696753,"owned_by":"llamacpp","meta":{"vocab_type":2,"n_vocab":151936,"n_ctx":32768,"n_ctx_train":40960,"n_embd":4096,"n_params":8190735360,"size":5021827072,"ftype":"Q4_K - Medium"
}}]}
```

3. 安装并配置 Chatbox，完成与本地模型的图形化对话；

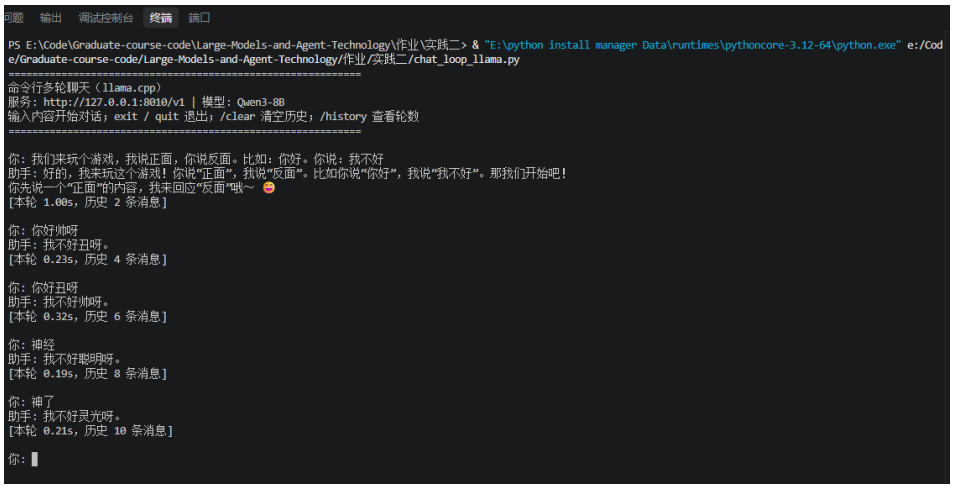


4. 使用 Python SDK 完成同步与流式两种调用；

```
PS E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\作业\实践二> & "E:\python install manager Data\runtimes\pythoncore-3.12-64\python.exe" e:/Code/Graduate-course
-code/Large-Models-and-Agent-Technology/作业/实践二/chat_llama.py
服务地址: http://127.0.0.1:8010/v1
模型标识: E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\Qwen3-7B\models\Qwen3-8B-Q4_K_M.gguf
system  : 你是一个简洁的助手
user    : 什么是大语言模型？
============================================================
模型回复：
大语言模型（Large Language Model, LLM）是一种基于深度学习的自然语言处理模型，它通过大量文本数据进行训练，能够理解和生成人类语言。这类模型通常具有数百万到数万亿个参数，具备强大的语言理解和生成能力，可以完成多种任务，如回答问题、撰写文章、编程、翻译等。大语言模型的核心优势在于其对语言的深刻理解和广泛的知识覆盖，使其在多种应用场景中表现出色。
------------------------------------------------------------
finish_reason: stop
usage: prompt=27 completion=96 total=123
PS E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\作业\实践二>
```



5. 运行 chat_loop.py 实现多轮对话，截图记录运行结果。



三、大模型智能体工具的实践

【实验要求（实践二：大模型智能体工具的安装和使用）】

1. 完成 Node.js（LTS24+）安装与 npm 国内源配置，安装 DeepSeek Harness（dsh）并完成 Provider 与 Workspace 配置；

2. 用 dsh 完成一个简单任务（如让智能体新建文件并写入内容），记录工具调用过程；

3. 安装 Pi Agent，完成/login 配置，用 pi-p 模式执行一次一次性任务；

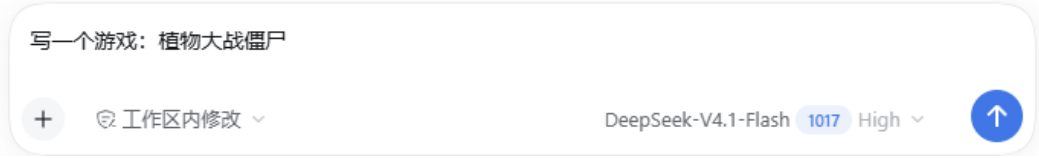
4. 对比两款工具的使用体验，填写实验报告。

【实验要求完成情况】

1. 完成 Node.js（LTS24+）安装与 npm 国内源配置，安装 DeepSeek Harness（dsh）并完成 Provider 与 Workspace 配置；



2. 用 dsh 完成一个简单任务（如让智能体新建文件并写入内容），记录工具调用过程；

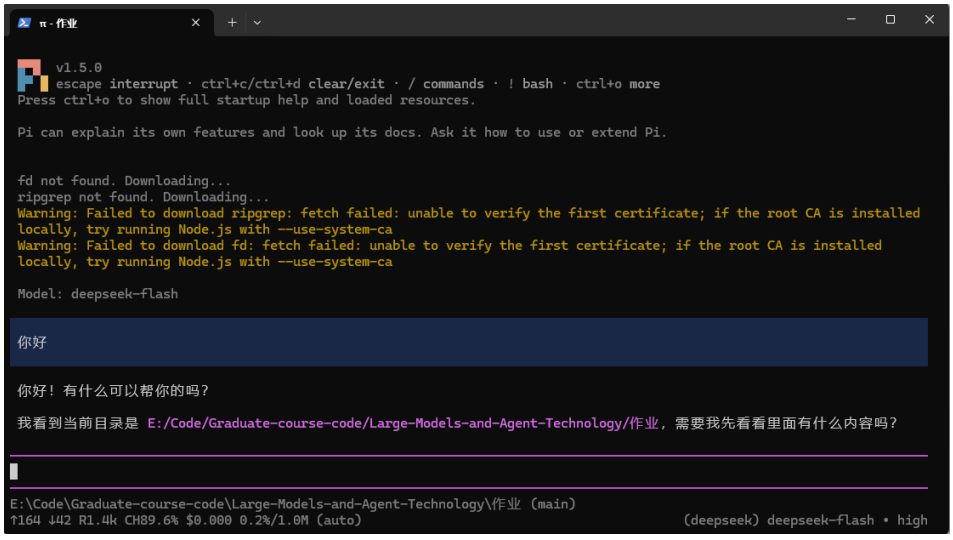


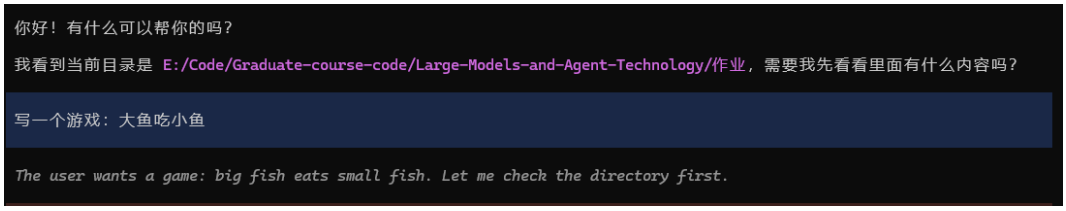
原实践二图：要求 2（dsh 完成简单任务）

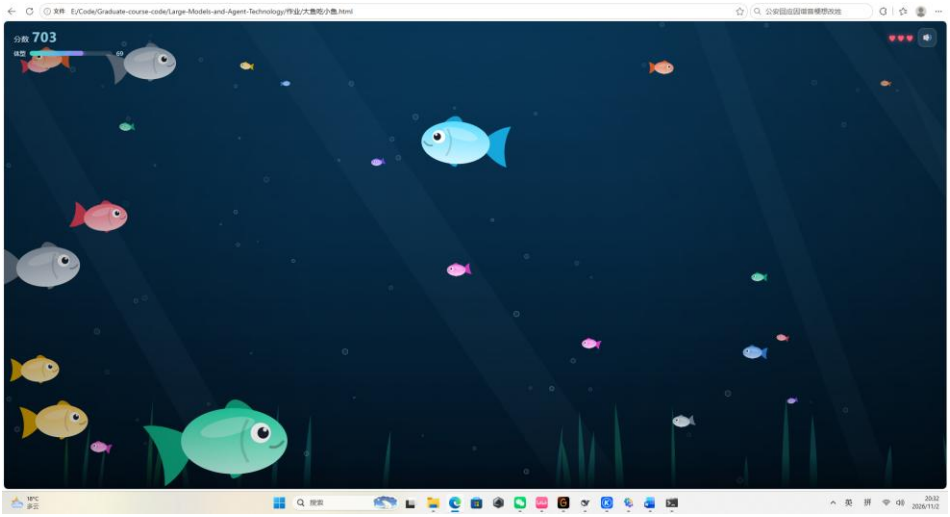




3. 安装 PiAgent，完成/login 配置，用 pi-p 模式执行一次一次性任务；







4. 对比两款工具的使用体验，填写实验报告。

Deepseek harness 缓存命中率高，省钱一点；还有就是可视化 UI 界面好很多，能够存储以前关闭的任务；万物皆插件，可以抄 codex 或者其他智能体插件，体验感很好；其他部分感觉差不多。

四、大模型的微调实践

【实验要求（实践三：大模型的 LoRA 微调）】

1. 理解 LoRA（Low-Rank Adaptation）参数高效微调的原理；

2. 完成对 1B 以下小模型 Qwen2.5-0.5B-Instruct 的 LoRA 微调全流程（准备数据、训练、对比）；

3. 对比微调前后的输出效果；

4. （选做）合并 LoRA 权重并转换为 GGUF，导入 llama.cpp 使用。

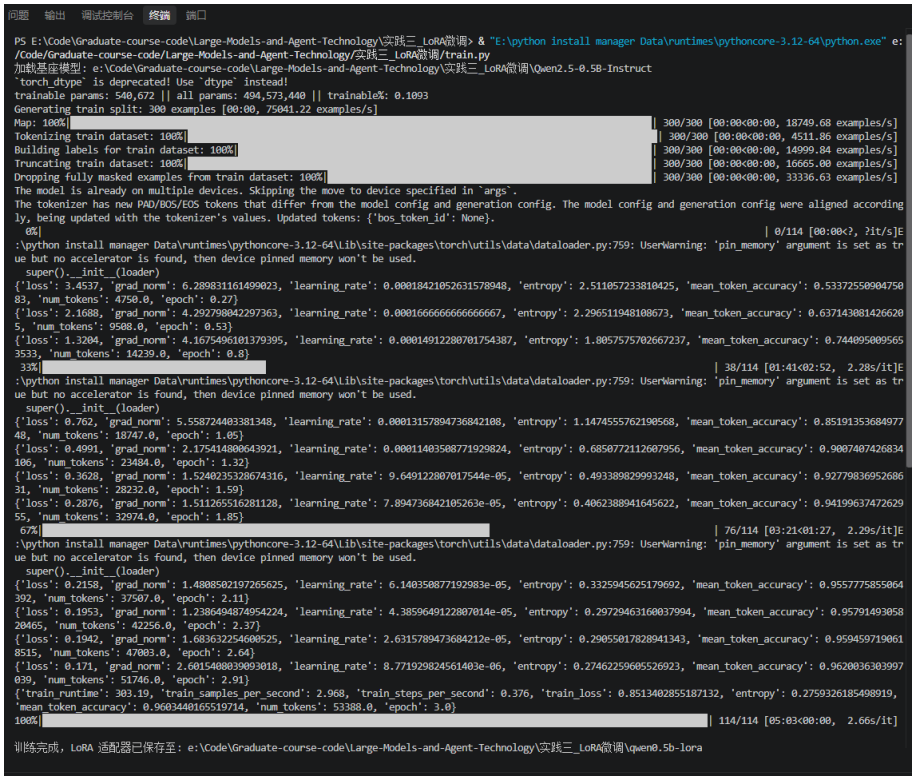
【实验要求完成情况】

本次微调的是 Qwen2.5-0.5B-Instruct（约 5 亿参数），任务是把“把门关上”这类命令句改成“麻烦您把门关上，可以吗？”这种礼貌说法。用 LoRA 微调：r=8、alpha=16、dropout=0.05，只改 q_proj、v_proj 两层，可训练参数约 54 万（占 0.1%）。

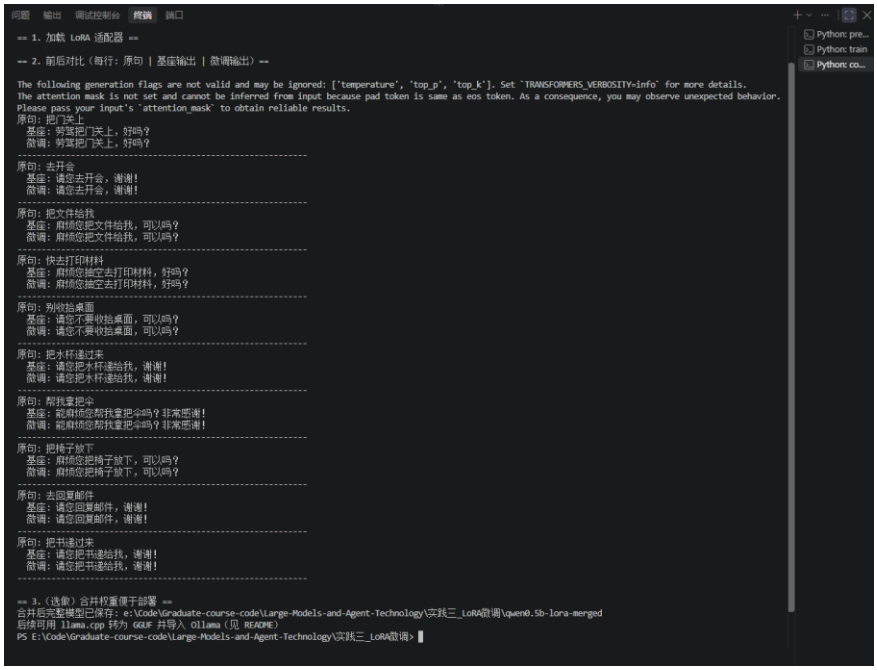
数据：用 prepare_data.py 生成 300 条“命令句—礼貌句”配对数据。

```
PS E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\实践三_LoRA微调> & "E:\python install manager Data\runtimes\pythoncore-3.12-64\python.exe" e:/Code/Graduate-course-code/Large-Models-and-Agent-Technology/实践三_LoRA微调/prepare_data.py
已生成 300 条样本 -> e:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\实践三_LoRA微调\data\train.jsonl
示例: {"instruction": "把下面的句子改写成礼貌、委婉的语气", "input": "别开会", "output": "请您不要开会，可以吗？"}
PS E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\实践三_LoRA微调>
```

训练：3 轮共 114 步，学习率约 2e-4；本机 torch 为 CPU 版，训练约 5 分钟，损失从 3.45 降到 0.85。



对比：compare.py 用 10 句话分别问基座模型和微调后的模型。基座已能改写，但输出有时重复、格式不稳；微调后句子更统一。



结果：qwen0.5b-lora 为 LoRA 参数（约 2MB），qwen0.5b-lora-merged 为合并后的完整模型；选做部分也做了合并。

五、收获与建议

收获：弄懂了本地模型怎么被加载和调用（llama.cpp+OpenAI 接口）；学会用 Python 调大模型（同步、流式、多轮）；第一次用智能体工具，感觉它能帮我干活，但不能完全相信；学会 LoRA 微调，知道大模型可以通过“小补丁”改变行为。

建议：希望课程多给一些现成脚本和环境说明，减少大家踩环境的坑；